**DIRECCIÓN DE EDUCACIÓN A DISTANCIA**

Diseño y Programación de Software Multiplataforma DPS941 (Virtual)

**DOCENTE:**
**Alexander Alberto Siguenza Campos**

**ACTIVIDAD 3**

Implementación final(móvil)

**PROYECTO:**
**Sonrisa Digital**

Sistema multiplataforma de gestión para la Clínica Dental Sonrisa Perfecta

**EQUIPO:**
**404 DPS Devs**

**INTEGRANTE (S)**

| Carné | Nombres completos |
|---|---|
| MV252134 | Milton Antonio Mayorga Vásquez |
| CP121738 | David Guillermo Cardona Pérez |
| CM210922 | Rene Gerardo Castillo Monterrosa |
| OA182314 | Mario Alejandro Orellana Andrade |
| RS221448 | Juan Diego Rodríguez Somoza |

**URL DE REPOSITORIO**

MiltonMV2025/DPS_PROYECTO

## I. Introducción

Sonrisa Digital es el sistema multiplataforma de gestión que el equipo 404 DPS Devs desarrolla para la Clínica Dental Sonrisa Perfecta. Está compuesto por un panel web, dirigido al personal de la clínica, y por una aplicación móvil dirigida a los pacientes. Este documento describe cómo se aplicaron dos herramientas complementarias en la parte móvil: Figma, con la que se diseñaron las interfaces, y React Native, con el que se programó la aplicación para Android e iOS (Meta Platforms, Inc., s. f.-b).

La aplicación móvil está implementada y funciona para el rol de paciente: permite iniciar sesión, consultar el estado de las citas, solicitar y cancelar citas, revisar el historial clínico y actualizar los datos de contacto (404 DPS Devs, 2026). Se construyó con Expo y React Native y se comunica con la misma API que utiliza el panel web, de modo que ambas plataformas comparten una única fuente de datos.

El documento se organiza de la siguiente manera. Primero se explica la aplicación de React Native en el proyecto; después se describe el trabajo de diseño realizado en Figma; luego se relacionan ambas herramientas, incluidas las diferencias entre lo diseñado y lo implementado; y, por último, se presentan los aportes, las limitaciones actuales y las conclusiones del equipo.

## II. Aplicación de React Native en el proyecto

React Native fue seleccionado para la aplicación móvil porque la clínica opera con dispositivos diversos: la recepción utiliza equipos Android, el odontólogo trabaja con una tableta iPad y los pacientes cuentan con teléfonos de distintas marcas y capacidades. Con una sola base de código en TypeScript es posible atender a esos usuarios sin mantener dos aplicaciones independientes (Román Gallardo et al., 2020).

La aplicación sustituye procesos que dependían de agendas físicas y registros manuales: el paciente consulta sus citas, solicita un horario y revisa su historial desde el teléfono. Todo el código móvil se encuentra en la carpeta mobile del repositorio del proyecto, junto al código del panel web (404 DPS Devs, 2026).

Técnicamente, React Native permite construir interfaces mediante componentes de React que se asocian con elementos nativos de cada plataforma; por ejemplo, View, Text, ScrollView y TextInput corresponden a vistas nativas de Android y de iOS (Meta Platforms, Inc., s. f.-b). La lógica de la aplicación se escribe en TypeScript y se apoya en el estado, los efectos y la composición de componentes propios de React (Madani, 2024; Román Gallardo et al., 2020).

### 2.1.Desarrollo para Android e iOS

La aplicación se creó con la versión 0.86 de React Native y la versión 19 de React. Desde la versión 0.76, la Nueva Arquitectura de React Native viene activada por defecto en los proyectos nuevos; esta reemplaza el puente asíncrono entre JavaScript y el código nativo por la interfaz JSI, que permite invocar métodos nativos directamente y sin serialización (Meta Platforms, Inc., s. f.-a). El proyecto no modifica esa configuración.

Contar con una base común tiene una consecuencia práctica: una regla de presentación, como la forma de mostrar una cita cancelada, se escribe una sola vez y se aplica en ambas plataformas. Sin embargo, compartir código no elimina la necesidad de probar en cada sistema. Hasta el momento las pruebas se realizaron en el emulador de Android; la verificación en dispositivos iOS está pendiente.

### 2.2.Uso de Expo y navegación

Se utilizó Expo (SDK 57) como capa de herramientas sobre React Native. Expo simplifica la configuración del proyecto, el arranque en desarrollo y el acceso a funciones del dispositivo mediante módulos como expo-secure-store y expo-notifications. Para la navegación se empleó Expo Router, que deriva las rutas de la estructura de archivos: cada archivo dentro de la carpeta app se convierte en una pantalla (Expo, s. f.-a).

La estructura resultante separa dos grupos de rutas. El grupo de autenticación contiene la pantalla de inicio de sesión; el grupo de la aplicación contiene las cinco pantallas del paciente, organizadas en una barra de pestañas inferior: Inicio, Mis citas, Agendar, Historial y Perfil. El diseño de este segundo grupo comprueba si existe una sesión activa y, si no la hay, redirige al inicio de sesión, de modo que ninguna pantalla con datos del paciente puede abrirse sin autenticarse.

### 2.3.Autenticación y manejo de la sesión

El inicio de sesión envía el correo y la contraseña al servidor, que responde con un token de acceso de corta duración y un token de renovación. El token de acceso se mantiene solo en memoria; el de renovación se guarda con expo-secure-store, que cifra los valores mediante el Keystore en Android y el Keychain en iOS (Expo, s. f.-c). Cuando el servidor indica que el token de acceso expiró, la aplicación solicita uno nuevo con el token de renovación y repite la petición una sola vez; si varias peticiones fallan al mismo tiempo, comparten una única renovación.

Al cerrar sesión, la aplicación elimina los tokens, borra los datos almacenados en caché y desregistra el dispositivo de las notificaciones. La limpieza de la caché evita que, si otro paciente inicia sesión en el mismo teléfono, vea información del anterior. Los mensajes de error

distinguen entre credenciales incorrectas y problemas de conexión, y las peticiones tienen un tiempo máximo de espera de 15 segundos para que la pantalla no quede cargando indefinidamente.

La protección efectiva de los datos reside en el servidor, que valida el token y el rol en cada petición. Ocultar una pantalla en la aplicación mejora la presentación, pero no sustituye esa comprobación.

### 2.4.Comunicación con el servidor y estados de las pantallas

La aplicación consume la API REST de Sonrisa Digital, la misma que utiliza el panel web, construida con Next.js sobre una base de datos MySQL y organizada en rutas, controladores, servicios y repositorios. Desde el móvil, las peticiones se realizan con axios y los datos del servidor se administran con TanStack Query, una biblioteca pensada para obtener, almacenar en caché y sincronizar el estado que proviene de un servidor remoto (TanStack, s. f.).

Cada pantalla contempla los estados que forman parte de la experiencia del usuario: carga, error con opción de reintentar, ausencia de datos y operación completada. Por ejemplo, cuando el paciente aún no tiene registros clínicos, la pantalla de historial explica que estos aparecerán después de completar una cita, y la cancelación de una cita bloquea únicamente la tarjeta afectada y muestra el resultado de la operación.

Una cita solicitada desde el teléfono aparece en la agenda de recepción porque ambas plataformas consultan los mismos datos. La aplicación no recibe cambios en tiempo real: la información se actualiza cuando las pantallas vuelven a consultarla.

### 2.5.Gestión de citas

La gestión de citas es la función central de la aplicación. La pantalla Agendar guía al paciente en cuatro pasos: elegir la fecha en un calendario, seleccionar la duración, escoger uno de los horarios disponibles de cada odontólogo y escribir el motivo de la consulta. El calendario no permite seleccionar días pasados y limita la búsqueda a los próximos 90 días. Al elegir un día o un horario, la pantalla se desplaza sola al paso siguiente para que el paciente no pierda el hilo en una pantalla pequeña.

Los horarios los calcula el servidor según la agenda de cada odontólogo, y la solicitud se envía con estado pendiente hasta que la clínica la confirma. Esta validación en el servidor es necesaria porque dos pacientes podrían elegir el mismo horario al mismo tiempo, algo que la interfaz por sí sola no puede evitar.

En Mis citas, el paciente ve cada cita con su fecha, hora, odontólogo, duración, motivo e indicador de estado, y puede cancelar las citas pendientes o confirmadas luego de confirmar la acción en un cuadro de diálogo. En Inicio, la tarjeta de próxima cita considera únicamente las citas confirmadas; las solicitudes pendientes se muestran aparte, en un contador, para no confundir una solicitud con una cita ya acordada.

### 2.6.Consulta del historial clínico

La pantalla Historial presenta cada atención en una tarjeta que incluye la fecha, el odontólogo, el motivo, la duración, el diagnóstico, el tratamiento, las observaciones, la receta y las recomendaciones. La información se solicita al servidor con la sesión del propio paciente, por lo que cada persona solo recibe sus registros. Las fechas y horas se muestran en un formato legible, por ejemplo, 20 oct 2026, 4:30 p. m.

El acceso a la información debe corresponder a los permisos de cada usuario. El historial es un dato sensible: su conservación y su protección dependen del servidor y de la base de datos, no de la pantalla del teléfono.

### 2.7.Perfil del paciente

El perfil muestra el nombre, el correo y el rol del paciente, y su fecha de nacimiento, que es de solo lectura. El paciente puede modificar su teléfono y sus alergias; al guardar, la aplicación actualiza los datos en el servidor y confirma el resultado en pantalla. El cierre de sesión se solicita mediante un cuadro de confirmación para evitar salidas accidentales.

### 2.8.Notificaciones y recordatorios

El proyecto contempla recordatorios 24 horas y una hora antes de cada cita, con el propósito de reducir la tasa de inasistencia mensual de la clínica, que ronda el 15 %. En la aplicación se implementó el registro del dispositivo: cuando la función se habilita mediante la variable EXPO_PUBLIC_ENABLE_PUSH, la aplicación solicita el permiso al usuario, obtiene el token del dispositivo con expo-notifications y lo registra en el servidor; al cerrar sesión, lo elimina (Expo, s. f.-b).

Esta función permanece desactivada por defecto porque, desde el SDK 53 de Expo, las notificaciones push remotas no funcionan en Expo Go y requieren una compilación de desarrollo (Expo, s. f.-b). Por ello, el envío de recordatorios de extremo a extremo aún no se ha comprobado y se presenta como trabajo pendiente. Tampoco se dispone de resultados que permitan afirmar que los recordatorios reducen el ausentismo; esa medición exige operar el sistema en la clínica durante un periodo.

### 2.9.Alcance de la aplicación móvil

La aplicación móvil está orientada al paciente. Las funciones del personal, como la agenda, el expediente clínico y el inventario, se atienden desde el panel web. La lectura de códigos de barras o QR mediante la cámara se planteó como una posible función móvil para registrar movimientos de insumos, pero no forma parte de esta entrega.

## III. Aplicación de Figma en el proyecto

Figma se utilizó para representar las interfaces de Sonrisa Digital antes de programarlas. El archivo del equipo incluye el panel del paciente en versión móvil, los módulos administrativos del panel web y un flujo de navegación por roles. Su principal valor fue permitir discutir pantallas concretas y detectar campos faltantes o acciones confusas antes de invertir tiempo en el código (Sinclair, 2026).

Figma permite además crear prototipos, es decir, flujos interactivos que simulan cómo una persona recorrería las pantallas mediante conexiones entre un elemento, como un botón, y un marco de destino (Figma, Inc., s. f.-b). Esta capacidad resulta útil para comprobar si los usuarios entienden cómo avanzar entre pantallas y completar una tarea (Sinclair, 2026).

Otra herramienta relevante es Auto Layout, que organiza automáticamente los elementos de un marco según el contenido, el espaciado y la alineación (Figma, Inc., s. f.-a; Sinclair, 2026). Sirve para revisar casos como nombres largos, tarjetas con más texto o pantallas de distintas dimensiones; su uso sistemático en todos los marcos queda como mejora para las próximas iteraciones del diseño.

### 3.1.Definición del lenguaje visual

El diseño establece una identidad visual basada en fondos claros, botones azules, tarjetas y una barra de navegación inferior. El azul dental definido en Figma es #007BFF y se aplica a las acciones principales. El criterio central es la consistencia: cuando un botón principal conserva la misma apariencia en varias pantallas, el usuario reconoce con mayor facilidad dónde debe actuar (Tidwell et al., 2024).

Los estados de las citas se distinguen con color y con texto: verde para las completadas, rojo para las canceladas y naranja para las pendientes. Como cada estado incluye su etiqueta, el significado no depende únicamente del color.

### 3.2.Diseño del inicio de sesión

La pantalla de acceso reúne el logo de la clínica, los campos de correo y contraseña, el botón de inicio de sesión y enlaces para recuperar la contraseña o registrarse. Figma permite analizar la jerarquía de estos elementos: la acción principal debe identificarse de inmediato y las opciones complementarias deben permanecer accesibles (Gómez Delgado, 2024).

Durante la implementación se añadieron situaciones que el diseño no representaba: campos vacíos, credenciales incorrectas, solicitud en proceso y falta de conexión.

### 3.3.Diseño del panel del paciente

El panel del paciente concentra la información relevante al entrar a la aplicación: un saludo personalizado, una tarjeta destacada con la próxima cita, una lista de citas anteriores y un acceso para agendar una nueva. Destacar la próxima cita responde al propósito de la aplicación, porque el paciente puede saber de inmediato cuándo debe asistir.

La barra inferior del diseño contiene Inicio, Mis Citas, Tratamientos, Mensajes y Ajustes. En la implementación esa barra se ajustó a las funciones efectivamente construidas, como se detalla en la Tabla 2. La sección Mensajes no tenía una función definida en el diseño, por lo que no se programó.

### 3.4.Diseño del calendario y la reserva

La pantalla Agendar Cita se diseñó como una secuencia visual: elegir un día, seleccionar un horario, introducir información opcional y confirmar. Los cambios de color y una marca de selección permiten al paciente ver qué opción eligió antes de enviar la solicitud.

En la implementación esa secuencia se convirtió en cuatro pasos numerados (fecha, duración, horario y motivo) y se agregaron los estados que el diseño no mostraba: ausencia de horarios, error al consultar la disponibilidad y solicitud enviada. Además, el horario elegido se resalta junto con el nombre del odontólogo.

### 3.5.Diseño del historial clínico

El diseño organiza el historial en tarjetas que resumen cada consulta e incluyen miniaturas de radiografías y de otras imágenes. La implementación actual muestra el contenido textual de cada atención; las miniaturas no forman parte de esta entrega. Cuando se incorporen, será necesario verificar la calidad de las imágenes, la rapidez de carga y la seguridad de los archivos, propiedades que el diseño por sí solo no acredita.

### 3.6.Diseño del perfil

La pantalla de perfil se diseñó para reunir datos personales, preferencias de notificación, cambio de contraseña, idioma, ayuda y cierre de sesión. El diseño presentaba algunos datos

como de solo visualización y mencionaba, a la vez, un botón para guardar cambios, por lo que faltaba definir qué campos se editan. La implementación resolvió esa definición: el nombre, el correo y la fecha de nacimiento son de solo lectura, y el teléfono y las alergias son editables. Las preferencias de notificación, el cambio de contraseña, el idioma y la ayuda quedan pendientes.

### 3.7.Diseño de las interfaces del personal

Figma también se utilizó para diseñar las interfaces administrativas: agenda, pacientes, inventario, reportes, usuarios y proveedores. Estas pantallas corresponden al panel web y no a la aplicación móvil; haberlas diseñado en Figma no implica que deban desarrollarse con React Native. Su presencia en el mismo archivo ayuda a entender cómo se relaciona el trabajo del paciente con el de la clínica: una solicitud hecha desde el teléfono tiene su representación en la agenda administrativa.

## IV. Relación entre Figma y React Native

La relación entre ambas herramientas está en el paso del diseño a la implementación: Figma describe la apariencia y el recorrido esperado, y React Native programa la interacción móvil y conecta las pantallas con los datos. La Tabla 1 resume cómo se materializó cada elemento diseñado.

**Tabla 1**

*Correspondencia entre el diseño en Figma y la implementación en React Native*

| Elemento | Diseño en Figma | Implementación con React Native |
|---|---|---|
| Inicio de sesión | Logo, campos de correo y contraseña, botón y enlaces | Pantalla con validación de campos, mensajes de error y sesión persistente mediante tokens |
| Inicio | Tarjeta destacada de próxima cita y lista de citas anteriores | Tarjeta de próxima cita (solo confirmadas) y contadores de solicitudes pendientes y citas confirmadas |
| Calendario y reserva | Días y horarios seleccionables e información opcional | Calendario propio, duración, horarios por odontólogo desde la API y solicitud en estado pendiente |

| Elemento | Diseño en Figma | Implementación con React Native |
|---|---|---|
| Mis citas | Estados diferenciados por color | Lista con indicador de estado y cancelación con confirmación |
| Historial | Tarjetas con miniaturas de imágenes | Tarjetas con diagnóstico, tratamiento, observaciones, receta y recomendaciones |
| Perfil | Datos personales y preferencias | Edición de teléfono y alergias; cierre de sesión con confirmación |
| Recordatorios | Avisos 24 horas y una hora antes de la cita | Registro del dispositivo para notificaciones; envío pendiente de validar |
| Inventario móvil | Flujo por detallar para el personal | No incluido; se atiende desde el panel web |

*Nota.* Elaboración propia a partir del archivo de Figma y de la aplicación implementada.

La correspondencia no consiste únicamente en copiar colores o posiciones, sino en conservar el significado de cada acción. Un botón diseñado para confirmar una cita debe informar cuándo está disponible, cuándo se está procesando y cuándo terminó; por eso los botones de la aplicación muestran el texto Cargando y se deshabilitan mientras la solicitud está en curso.

Durante la implementación surgieron diferencias entre lo diseñado y lo construido, que se resumen en la Tabla 2. Lo conveniente es actualizar el archivo de Figma para que vuelva a reflejar la aplicación; mientras tanto, la aplicación en funcionamiento es la referencia.

**Tabla 2**

*Diferencias entre el diseño en Figma y la aplicación implementada*

| Aspecto | Diseño en Figma | Aplicación implementada |
|---|---|---|
| Barra de navegación | Inicio, Mis Citas, Tratamientos, Mensajes y Ajustes | Inicio, Mis citas, Agendar, Historial y Perfil |
| Color primario | #007BFF | #33A7DC |
| Inicio de sesión | Enlaces para recuperar la contraseña y registrarse | Correo, contraseña y botón de inicio de sesión |
| Perfil | Notificaciones, contraseña, idioma y ayuda | Teléfono, alergias y cierre de sesión |
| Historial | Miniaturas de radiografías | Texto clínico sin imágenes |
| Reserva | Calendario, horarios e información opcional | Cuatro pasos, con duración y horarios por odontólogo |

*Nota.* Elaboración propia a partir del archivo de Figma y de la aplicación implementada.

## V. Aportes de ambas herramientas al proyecto

La principal contribución de Figma fue hacer visible la solución antes de programarla. Permitió discutir pantallas concretas y detectar si faltaban campos, si una acción resultaba confusa o si la navegación respondía a las necesidades del paciente.

React Native aportó una base de código compartida para Android e iOS y, junto con Expo, el acceso a funciones del dispositivo como el almacenamiento seguro y las notificaciones.

En conjunto, ambas herramientas ayudan a reducir las correcciones tardías, aunque el diseño por sí solo no las evita. Al revisar la aplicación en funcionamiento con datos de demostración, el equipo detectó, por ejemplo, que la tarjeta de próxima cita contaba solicitudes todavía pendientes y que el calendario permitía elegir fechas pasadas; ambos casos se corrigieron. Son problemas que un diseño estático no muestra y que solo aparecen al probar con datos reales.

También favorecen la continuidad entre las pantallas. Las tarjetas, los botones y los mensajes de error se implementaron como componentes comunes, lo que facilita el aprendizaje del usuario y el mantenimiento del software.

La reutilización entre la plataforma web y la aplicación móvil debe entenderse con precisión: ambas comparten la API y las reglas del servidor, pero los componentes visuales web no se trasladan directamente, por lo que se escribieron de nuevo para el teléfono.

## VI. Alcance actual, limitaciones y trabajo futuro

La aplicación se probó manualmente en el emulador de Android con datos de demostración. Las observaciones que el equipo identificó durante su revisión se corrigieron y se

verificaron en ese entorno, y el código pasa la verificación de tipos de TypeScript. No se han realizado pruebas en dispositivos iOS ni pruebas automatizadas, y la verificación en teléfonos físicos está pendiente.

Quedan pendientes, en primer lugar, las pantallas de registro y de recuperación de contraseña que aparecen en el diseño. En segundo lugar, la validación del envío de recordatorios, que requiere una compilación de desarrollo. En tercer lugar, la incorporación de imágenes al historial clínico. En cuarto lugar, las preferencias de notificación en el perfil y la validación de los datos que el paciente edita, como el teléfono. Por último, la personalización del ícono y de la pantalla de carga con la identidad de la clínica.

Finalmente, los beneficios esperados, como reducir las inasistencias, ahorrar tiempo de recepción o mejorar el registro de insumos, son objetivos que requieren evaluación. Para afirmar que se alcanzaron será necesario medir el funcionamiento del sistema después de ponerlo en uso en la clínica.

## VII. Conclusiones

Figma y React Native cumplieron funciones distintas y complementarias en Sonrisa Digital. Figma permitió definir y revisar las interfaces antes de programarlas, y React Native, junto con Expo, permitió convertir esas pantallas en una aplicación funcional para el paciente, conectada a la misma API que el panel web.

La aplicación implementada cubre el flujo principal del paciente: autenticarse, solicitar y cancelar citas, consultar el historial y mantener sus datos de contacto. Las diferencias con el diseño original, que se documentaron en este informe, muestran que el diseño es un punto de

partida que se ajusta al implementar y probar. El equipo continuará con las funciones pendientes y con la actualización del archivo de Figma.

## Referencias

404 DPS Devs. (2026). *Sonrisa Digital* [Software]. GitHub. https://github.com/MiltonMV2025/DPS_PROYECTO

Expo. (s. f.-a). *Introduction to Expo Router*. Expo Documentation. Recuperado el 20 de octubre de 2026, de https://docs.expo.dev/router/introduction/

Expo. (s. f.-b). *Notifications*. Expo Documentation. Recuperado el 20 de octubre de 2026, de https://docs.expo.dev/versions/latest/sdk/notifications/

Expo. (s. f.-c). *SecureStore*. Expo Documentation. Recuperado el 20 de octubre de 2026, de https://docs.expo.dev/versions/latest/sdk/securestore/

Figma, Inc. (s. f.-a). *Guide to auto layout in Figma*. Figma Help Center. Recuperado el 20 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/360040451373-Guide-to-auto-layout-in-Figma

Figma, Inc. (s. f.-b). *Guide to prototyping in Figma*. Figma Help Center. Recuperado el 20 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/360040314193-Guide-to-prototyping-in-Figma

Gómez Delgado, J. (2024). *Diseño de interfaces para la web actual: De la estructura de la interfaz a la usabilidad*. ESIC Editorial. https://www.esic.edu/editorial/diseno-de-interfaces-para-la-web-actual

Madani, H. (2024). *React: Desarrolle el front end de sus aplicaciones web y móviles con JavaScript*. Ediciones ENI. https://www.ediciones-eni.com/libro/react-desarrolle-el-front-end-de-sus-aplicaciones-web-y-moviles-con-javascript-9782409046049

Meta Platforms, Inc. (s. f.-a). *About the New Architecture*. React Native. Recuperado el 20 de octubre de 2026, de https://reactnative.dev/architecture/landing-page

Meta Platforms, Inc. (s. f.-b). *Core components and native components*. React Native. Recuperado el 20 de octubre de 2026, de https://reactnative.dev/docs/intro-react-native-components

Román Gallardo, A., Flores Cortés, C. A., y Herrera Morales, J. R. (2020). *Programación de aplicaciones móviles con React Native*. Editorial BUK. https://www.buk.com.mx/9786079878214/description

Sinclair, N. A. A. (2026). *La guía completa de Figma: Vol. 1. Guía paso a paso para principiantes en el diseño de interfaces de usuario, creación de prototipos interactivos, creación de sistemas de diseño, colaboración con equipos y entrega de proyectos profesionales de UI/UX*. Publicación independiente.

TanStack. (s. f.). *Overview*. TanStack Query. Recuperado el 20 de octubre de 2026, de https://tanstack.com/query/latest/docs/framework/react/overview

Tidwell, J., Brewer, C., y Valencia, A. (2024). *Diseño de interfaces* (3.ª ed.). O'Reilly Media. https://www.oreilly.com/library/view/diseno-de-interfaces/9781098181611/